EECS 498/CSE 598: Zero-Knowledge Proofs
Winter 2026
Lecture 16

Paul Grubbs

paulgrub@umich.edu

Beyster 4709

Agenda for this lecture

- Announcements
- Review of what a CPU is
- Why CPU-model frontends?
- Key ideas of CPU-model frontends
- Memory accesses

Agenda for this lecture

- Announcements
- Review of what a CPU is
- Why CPU-model frontends?
- Key ideas of CPU-model frontends
- Memory accesses

Announcements

- Project 3 online, **due 3/24**
- Theory track problem set 1 online, **due 3/24**
- The plan is to release P4 and theory pset 2 tomorrow.
    - Note there will be no P5 nor a final exam. These will be the last assignments of the semester.
- 598 research track: **project proposal due Friday 3/27**
- All work for this course must be completed by our final exam date, April 27th

Agenda for this lecture

- Announcements
- Review of what a CPU is
- Why CPU-model frontends?
- Key ideas of CPU-model frontends
- Memory accesses

What is a CPU, anyway?

Random Access Machine

- Main memory: S cells of fixed width "w"
- Special memory cells "registers"
- A set of instructions
  - memory read/write
    read A FF81
  - Operations arithmetic, boolean
- Special registers "flags" + Program counter

Agenda for this lecture

- Announcements
- Review of what a CPU is
- Why CPU-model frontends?
- Key ideas of CPU-model frontends
- Memory accesses

Drawbacks of ASIC model

- program fixed
- memory accesses (SSA
- CPU proving
- Need to rewrite "normal" programs

Agenda for this lecture

- Announcements
- Review of what a CPU is
- Why CPU-model frontends?
- Key ideas of CPU-model frontends
- Memory accesses

Key ideas

Performance questions: # steps of program
memory used
efficiency of instruction embedding into RICS

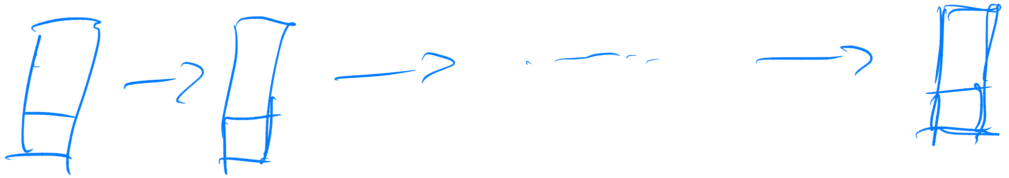
Define:
ISA - instructions
memory semantics

P x

Compile P to ISA P'

"bytecode": opcode sequence and memory accesses

Next:
Run P' on x to get transcript T



Key ideas

Goal: Prove knowledge of T that is correct for P' on X, output Y

What is in the transcript? $t :=$ length of program

Naively, everything. Too expensive!

1. starts with public input X
2. list of tuples
   (timestamp, list of registers, read or write info)

Key ideas

- How to check consistency?
  1. correctness of memory accesses
  2. correctness of CPU "step" function on registers

RICS "switch"

F(op reg_1 reg_2, st)
    if op = add:
        ...
    elif op = mul:
        ...
    elif op: XOR
        ⋮

Prove (1)?

Agenda for this lecture

- Announcements
- Review of what a CPU is
- Why CPU-model frontends?
- Key ideas of CPU-model frontends
- Memory accesses

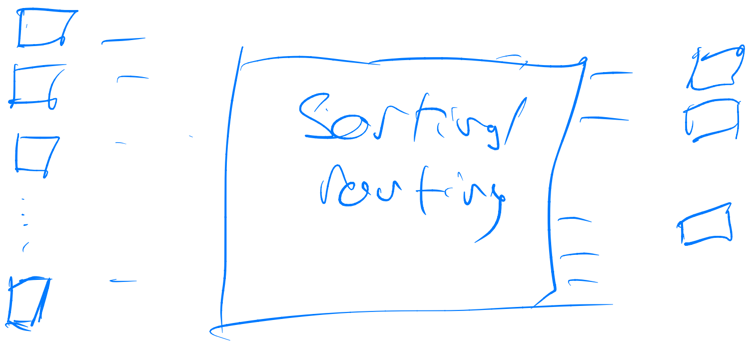
Checking memory consistency

If reads/writes ordered by memory location, break ties by timestamp

check consistency:

For each adj. pair:
if addresses are equal
and the latter is read,
check val read is same as former val

Sorting/routing network



Merkle tree

memory Finger-printing